RAG Argument Analysis Pipeline

Operational Guide — Implementation and Testing

April 2026 Version

This document describes the two pipelines (OpenAI and local Ollama) and the implementation steps for a new installation. It is designed to be copied into a new terminal window.

1. Overview

The pipeline is organized into two independent branches that share the same logic but differ in the LLM engine and data handling.

Dimension	OpenAI Path	Local (Ollama) Path
Generative LLM	gpt-4.1-mini or gpt-4.1	qwen2.5:14b, deepseek-r1:14b, gemma3:12b…
Embeddings	text-embedding-3-large (API)	paraphrase-multilingual-mpnet-base-v2 (local)
API key required	Yes (.env)	No
Data transmitted	Paragraphs + corpus passages	None — everything stays local
Cost	~$0.10–0.50 / batch run	Zero
RAM required	Minimal	16 GB (14B models)
Batch quality 09/10	Very good	Good (deepseek-r1 recommended)
A/B/C/D segment quality	Excellent	Good to very good depending on the model

⚠ Both tracks share the same scripts 11, A, B, C, D (suffix _local for Ollama). Scripts 01, 02, 03 are identical in both tracks.

2. Prerequisites common to both tracks

2.1 Python environment

```
conda create -n rag_historien python=3.11
conda activate rag_historien
pip install -r requirements.txt
```

⚠ *On macOS, if WeasyPrint fails: brew install pango cairo*

2.2 Project Folder Structure

```
MyProject/
├── .env                    ← OPENAI_API_KEY=sk-... (OpenAI path only)
├── rag_config_local.py     ← local configuration
├── config_00.py            ← OpenAI configuration
├── [all .py scripts]
├── pdfs/                   ← your corpus PDFs (path B without Zotero)
├── extracted_text/         ← generated by 01 and 02
├── vector_store/           ← generated by 03 (OpenAI path)
├── vector_store_local/     ← generated by 03_local (Ollama path)
└── results/              ← all outputs
```

2.3 Local method — Ollama

```
# Install Ollama: https://ollama.com
ollama serve                    # run in the background
call pull qwen2.5:14b         # default template
# Recommended alternatives for 09/10:
ollama pull deepseek-r1:14b     # best at structured reasoning
ollama pull gemma3:12b          # very good in French
If 32G
ollama pull qwen2.5:32b       # best all-rounder
ollama pull deepseek-r1 :32b. # recommended for 9/10
ollama pull gemma3 :27b.      # best in French
```

⚠ *Edit LLM_MODEL in rag_config_local.py to change the model across the entire pipeline.*

3. OpenAI Pipeline

3.1 Option A — with Zotero (formatted bibliographic references)

Use this path if your corpus is managed in Zotero. Citations (author, title, year) will be automatically inserted into all reports.

Step	Command
Export Zotero	`File → Export Library → CSV Format (All Columns)`
Convert to HTML	`python 00a_html_to_pdf.py --csv IAHistoire.csv`
Import Zotero	`python 00_zotero_import.py --csv IAHistoire.csv`
Check	`ls pdfs/ | wc -l   # should display ~287`
Text extraction	`python 01_extract_text.py`
Chunking	`python 02_chunk_corpus.py`
Embeddings	`python 03_build_embeddings.py`
Rag Query	`Python 04_rag_query.py`
Assisted Writing	`Python 05_rag_write.py -paragraph my_paragraph.txt`
Enriched mapping	`python 06_map_enrich.py --manuscript manuscript.txt`
Critical mapping	`python 07_map_critique.py --manuscript manuscript.txt`
Batch argumentation	`python 09_map_argumentation_openai.py`
Perelman batch	`python 10_map_perelman_openai.py`
Priority Areas	`python 11_priority_zones.py`
Segment A analysis	`python A_toulmin_segment.py`
Segment B analysis	`python B_perelman_segment.py`
Segment C analysis	`python C_rst_segment.py`
Cross-analysis D	`python D_cross-synthesis.py`
Recommendations E	`Python E_recommendations_openai.py`

3.2 Option B — without Zotero (pdfs/ folder)

Place the PDFs in the pdfs/ folder and skip steps 00a and 00_zotero_import. The references displayed will be the filenames.

Step	Command
Place PDFs	`Copy your PDFs to pdfs/`
Text extraction	`python 01_extract_text.py`
Chunking	`python 02_chunk_corpus.py`
Embeddings	`python 03_build_embeddings.py`
Same sequence	`→ Steps 06 through D as in Path A`

3.3 Updating Zotero (Path A — Incremental)

When you have added references in Zotero and want to integrate them without rebuilding the entire index:

Step	Command
Re-export CSV	`File → Export Library → CSV`
New HTML	`python 00a_html_to_pdf.py (optional, if new HTML)`
Update index	`python 00b_zotero_update.py --csv IAHistoire.csv`
Rerun analyses	`→ Steps 06 through D directly`

4. Local pipeline (Ollama — zero cost, zero exposure)

The local pipeline is identical to the OpenAI pipeline, with two differences: the embedding model is SentenceTransformers (local) and the generation LLM is served by Ollama. The vector_store_local/ directory is separate from vector_store/ — the two indexes coexist.

> ⚠ *The Ollama server must be running in the background before launching any _local scripts.*

4.1 Local Path A — with Zotero

Step	Command
Zotero → CSV + HTML	`python 00a_html_to_pdf.py && python 00_zotero_import.py`
Extraction + chunking	`python 01_extract_text.py && python 02_chunk_corpus.py`
Local embeddings	`python 03_build_embeddings_local.py`
Rag query	`python 04_rag_query_local.py`
Local enrichment	`python 05_compare_local.py`
Assisted writing	`python 05_compare_local.py`
Enriched mapping	`python 06_map_enrich_local.py --manuscript manuscript.txt`
Critical mapping	`python 07_map_critique_local.py --manuscript manuscript.txt`
Batch Argumentation	`python 09_map_argumentation_local.py`
Perelman batch	`python 10_map_perelman_local.py`
Priority areas	`python 11_priority_zones.py  (identical, zero cost)`
Segment A analysis	`python A_toulmin_segment_local.py`
Segment B analysis	`python B_perelman_local_segment.py`
Segment C analysis	`python C_rst_local_segment.py`
Cross-synthesis D	`python D_cross-synthesis_local.py`
Recommendations	*`python`* `E_recommendations_local.py`

4.2 Changing the local LLM model

Just one line to change in rag_config_local.py — the entire pipeline updates automatically.

Model	RAM	Recommended usage
`deepseek-r1:14b`	16 GB	Scripts 09, 10, A, B, C — structured reasoning (Toulmin/Perelman/RST)
`gemma3:12b`	16 GB	Scripts 09, 10, A, B, C — excellent in academic French
`qwen2.5:14b`	16 GB	Default — good overall balance across all scripts
`mistral:7b-instruct`	8 GB	Scripts 04, 05, 06, 07 — if RAM < 16 GB
`qwen2.5:7b`	8 GB	If RAM < 16 GB — acceptable quality for 06/07

```
# In rag_config_local.py — change a single line:
LLM_MODEL = "deepseek-r1:14b"   # or gemma3:12b, mistral:7b-instruct…
```

5. Levels of data sovereignty

The pipeline offers three levels of control over manuscript data exposure.

Level	What remains local	What is transmitted
`1 — All local (local path A or B)`	Corpus, manuscript, embeddings, analyses	Nothing — zero external exposure
`2 — Hybrid (recommended)`	Corpus, batch manuscript (09/10 locally) embeddings (SentenceTransformers)	Consciously selected segments (scripts A/B/C via OpenAI)
`3 — Fully cloud-based (via OpenAI)`	Embeddings if SentenceTransformers	Entire manuscript, paragraph by paragraph (06/07/09/10)

⚠ Level 1 is the recommended architecture for unpublished manuscripts: local batch (signals) + OpenAI segments on selected areas (in-depth analysis permitted).

6. Testing procedure on a minimal corpus

To validate the installation before running it on the full corpus, use a test corpus of 2–3 PDFs and a manuscript of 5–10 paragraphs.

6.1 OpenAI test

Test	Command and expected result
API key	`python -c "import openai; print('OK')"`
Embeddings API	`python 03_build_embeddings.py → vector_store/ created`
Query	`python 04_rag_query.py → response in French`
Review	`python 07_map_critique.py → critique.json + critique.md`
Argumentation	`python 09_map_argumentation_openai.py → argumentation_*.json`
Perelman	`python 10_map_perelman_openai.py → perelman_*.json`
Zones	`python 11_priority_zones.py → zones_*.md`
Segment A	`python A_toulmin_segment.py → toulmin_*.md`

6.2 Local path test

Test	Command and expected result
Ollama active	`curl http://localhost:11434 → {"ollama":"is running"}`
Template available	`ollama list → qwen2.5:14b (or other) listed`
Local embeddings	`python 03_build_embeddings_local.py → vector_store_local/ created`
Local query	`python 04_rag_query_local.py → streamed response`
Local review	`python 07_map_local_review.py → review.json + review.md`
Local argumentation	`python 09_map_argumentation_local.py → argumentation_*.json`
Local Segment A	`python A_toulmin_segment_local.py → toulmin_*.md`

6.3 Verifying Zotero References

After running 00_zotero_import.py and 03_build_embeddings.py, verify that the references appear in the output:

```
python -c "
import json
m = json.load(open('vector_store/metadata.json'))
print(m[0].get('ref_courte', 'ABSENT'))  # should display Author (Year)
"
```

7. Common Issues and Solutions

Symptom	Probable Cause	Solution
`All scores are 0.50 in the reports`	LLM did not follow the X/10 format	Robust `extract_scores()` already in place — verify that the correct file is being used
`'Not available' sections in summary D`	Separators ——— not generated by the LLM	Tolerant `extract_section_llm()` already in place
`UnboundLocalError on THRESHOLD_ROBUST_LOW`	Old version of 11	Replace with the corrected 11_zones_prioritaires.py
`Ollama inaccessible`	ollama serve not running	ollama server (running in the background)
`Truncated responses in A/B/C`	MAX_TOKENS too low	Increase to 4500 (A/B) or 5000 (C) at the beginning of the script
`WeasyPrint fails (macOS)`	Pango is missing	brew install pango cairo
`Short ref missing in the output`	metadata_refs.json missing or 03 not rerun	Rerun 03_build_embeddings.py after 00_zotero_import.py
`PDF not found for a Zotero item`	Incorrect storage path	Check ZOTERO_STORAGE in 00_zotero_import.py

8. Complete list of scripts

8.1 Scripts common to both workflows

Script	Path	Role
00a_html_to_pdf.py	A (Zotero)	Converts Zotero HTML snapshots to PDF (WeasyPrint)
00_zotero_import.py	A (Zotero)	Initial import: pdfs/ + metadata_refs.json
00b_zotero_update.py	A (Zotero)	Incremental index update (without rebuild)
01_extract_text.py	A + B	Text extraction from PDFs
02_chunk_corpus.py	A + B	Chunking with overlap
11_priority_zones.py	A + B	09×10 grid, zero-cost, navigation

8.2 OpenAI path scripts

Script	Input	Output
03_build_embeddings.py	chunks.json	vector_store/ + metadata.json
04_rag_query.py	Interactive query	Summary response (console)
05_rag_write.py	Writing instructions	redaction_*.txt
06_map_enrich.py	manuscript.txt	map_{ts}.json + map_{ts}.md
07_map_critique.py	manuscript.txt	review.json + review.md
08_map_visualization.py	Map__{ts}.json	html
09_map_argumentation_openai.py	map_{ts}.json	argumentation_{ts}.json + .md
10_map_perelman_openai.py	map_{ts}.json	perelman_{ts}.json + .md
A_toulmin_segment.py	Interactive segment	toulmin_{ts}.md + .json
B_perelman_segment.py	Interactive segment	perelman_seg_{ts}.md + .json
C_rst_segment.py	Interactive segment	rst_{ts}.md + .json
D_cross-synthesis.py	JSON A+B+C	cross-synthesis_{ts}.md
E_openai_recommendations.py	cross-synthesis_{ts}.md	recommendations_{ts}.md
09_viz_argumentation.py	argumentation_{ts}.json	HTML
10_viz_perelman.py	perelman_{ts}.json	HTML
Semantic_search.py	Interactive query	FAISS results (console)
Recalculate_scores	Recalculates and corrects the enrichment scores in enrich.json in case of parsing failure — maintenance utility, not part of the normal workflow	Maintenance utility to use if the default score values (0.5) appear too frequently in enrich.json

8.3 Local pipeline scripts (Ollama)

Local script	OpenAI equivalent	Difference

03_build_embeddings_local.py	03_build_embeddings.py	Local SentenceTransformers → vector_store_local/
04_rag_query_local.py	04_rag_query.py	Ollama instead of OpenAI
05_rag_write_local.py	05_rag_write.py	Ollama, temperature 0.3
06_map_enrich_local.py	06_map_enrich.py	Ollama
07_map_critique_local.py	07_map_critique.py	Ollama
09_map_argumentation_local.py	09_map_argumentation_openai.py	Ollama, LLM_MODEL from rag_config_local
10_map_perelman_local.py	10_map_perelman_openai.py	Ollama, LLM_MODEL from rag_config_local
A_toulmin_segment_local.py	A_toulmin_segment.py	Ollama, time estimation instead of cost
B_perelman_segment_local.py	B_perelman_segment.py	Ollama
C_rst_segment_local.py	C_rst_segment.py	Ollama
D_cross-synthesis_local.py	D_cross-synthesis.py	Ollama
E_recommendations_local.py	E_recommendations_openai.py	Ollama